1. (10pt) **Name:** _____

INSTRUCTIONS: Carefully read each question, and write the answer in the space provided. If answers to free response questions are written obscurely, zero credit will be awarded. The correct answer to a free response question with a short answer (i.e., one word or phrase) will never contain any significant words used in the question itself (i.e., "crossword rules"). You are permitted to use one 8.5x11 inch sheet of paper (double-sided) containing **hand-written** notes; all other aids (other than your brain) are forbidden. Questions may be brought to the instructor. You are required to turn in your notes with your exam; they will be returned to you with your graded exam.

You have **80 minutes** to complete the exam. You may turn in your exam as soon as you are finished. There are **800 points** available on the exam, and most questions are worth a number of points that is divisible by ten. These point values are a rough guide to how long I think you should spend on each question.

For **TRUE** or **FALSE** and multiple choice questions, circle your answer.

On free response questions only, you will receive **20%** credit for any question which you leave blank (i.e., do not attempt to answer). Do not waste your time or mine by making up an answer if you do not know. (Note though that most questions offer partial credit, so if you know part of the answer, it is almost always better to write something rather than nothing.)

You may rip out any page in this exam. If a page has questions on it, make sure your UCID is on it before you separate it from the rest of your exam.

To get credit for this question, you must:

- Print your name (e.g., "Martin Kellogg") in the space provided on this page.
- Print your UCID (e.g., "mjk76") in the top-right of **each** page of the exam with a question on it.

Contents (blanks for graders only):

Writing your name and UCID:	**10** / 10
Pages 2 and 3:	**190** / 190
Pages 4 and 5:	**270** / 270
Pages 6 and 7:	**30** / 30
Pages 8 and 9:	**190** / 190
Pages 10 and 11:	**110** / 110
Extra Credit:	**35** / 0
Total:	**835** / 800

I. Multiple Choice and Very Short Answer (190pts). In the following section, either circle your answer (possible answers appear in **bold**) or write a very short (one word or one phrase) answer in the space provided. No partial credit is possible in this section.

2. (15pt) Alice and Bob work on separate branches. Alice changes the function `formatDate` in `dates.ts` so that it returns `"2026-10-12"` instead of `"10/12/2026"`. Bob adds a new call to `formatDate` in `report.ts`, and his code relies on the old format. Neither of them edits the other's file. Git merges both branches into `main` without reporting a conflict. **TRUE** or **FALSE**: therefore, the merged program behaves correctly.

3. (20pt) A test calls `search("cat")` and passes if the URL `cats.example.com` appears somewhere in the returned list of results. What kind of comparator does this test use?

   A exact match

   B over-approximation

   C under-approximation

   D it has no comparator; it only has an oracle

4. (20pt) Which of these is a *verifiable non-functional* requirement for a library's book search?

   A "Search should be fast."

   B "Searching for an exact title shall list that book first."

   C "95% of searches shall return results within 200 ms."

   D "Search should be easy for new users."

5. (15pt) **TRUE** or **FALSE**: in a TypeScript project, the team should commit `dist/town.js`, which the TypeScript compiler generates from `src/town.ts`, to the team's repository.

6. (20pt) A user story reads: "As a driver, I want the turn-signal indicator to blink exactly every 1.5 seconds using a 555 timer chip, so that other drivers know I'm turning." Which INVEST property does this story most clearly violate?

   A Independent

   B Negotiable

   C Valuable

   D Testable

7. (30pt) Your front-end code calls a speech-to-text API that charges money for each request, and whose server-side implementation isn't finished yet. When you unit test the front end, what should you replace the API with? **a mock (also accept: mock object, fake, stub)**

8. (20pt) Which of these is the best *milestone* for a project?

   A The payment module is 80% complete.

   B Most of the known bugs have been fixed.

   C The login page has been demoed to the client.

   D The team has spent four weeks on the backend.

9. (20pt) Which statement about pair programming is best supported by research?

   A It reduces defects, and it reduces total development cost.

   B It reduces defects, but it increases total development cost.

   C It increases defects, but it reduces total development cost.

   D It has no measurable effect on defects or on development cost.

10. (30pt) A teammate writes a shell script that builds and deploys your team's app, and keeps it in their home directory. Instead, they should **commit** it, and then ask a teammate to **review** it. **Also accept for the first blank: check in, push, or add it to version control / the repository. Also accept for the second blank: code review, approve. Scoring (an exception to this section's no-partial-credit rule): 30 points if both blanks are correct, 10 points if only one is.**

A. linter	**B.** product manager	**C.** engineering manager	**D.** scrum master
E. waterfall process	**F.** opinionated	**G.** functional specification	**H.** *ad hoc* process
I. regression testing	**J.** premature optimization	**K.** over-engineered	**L.** integration testing
M. user story	**N.** formal specification	**O.** unit testing	**P.** hermetic build

II. Matching (160pts). This section contains a collection of terms discussed in class in an "Answer Bank" (choices **A.** through **P.**). Each question in this section describes a situation associated with an answer in the Answer Bank. Write the letter of the term in the Answer Bank that best describes each situation. Each answer in the Answer Bank will be used at most once.

11. (20pt) **B: product manager** Carmen schedules a meeting with an engineer to argue for a feature that she believes will particularly benefit certain customers.

12. (20pt) **P: hermetic build** After a customer encounters a bug in an old version, LaMonica can easily debug exactly the same binary in which the bug occurred.

13. (20pt) **I: regression testing** When opening a PR that fixes a bug reported by a customer, Cory includes the input that the customer supplied in their bug report.

14. (20pt) **F: opinionated** When selecting a formatting tool, Rosie chooses one with few options.

15. (20pt) **H: ad hoc process** Analilia's team spends more and more time reworking their system as it grows larger.

16. (20pt) **N: formal specification** Using a model of his distributed system, Joe catches a subtle concurrency error.

17. (20pt) **K: over-engineered** During a review, Justin notices that his coworker's design addresses a purely theoretical concern about what might need to change during maintenance.

18. (20pt) **G: functional specification** Nellie and her team's client agree on a document describing what the system she is building should do.

III. Short answer (110pts). Answer the questions in this section in at most three sentences.

The questions on this page refer to the following code:

```
1  const HAS_DIGIT = /[0-9]/;
2
3  function scoreStrength(password: string): boolean {
4    let score = 0;
5    if (password.length >= 12) {
6      score += 2;
7    } else if (password.length >= 8) {
8      score++;
9    }
10   if (HAS_DIGIT.test(password)) {
11     score++;
12   }
13   return score > 2;
14 }
```

19. This code has (at least) two different *kinds* of code-level design problems. For each kind, give one line where it occurs and explain how you would fix it. **The two kinds can be given in either order.**

    (a) (30pt) The first kind of problem: **Misleading name (line 3): "scoreStrength" suggests that the function returns a numeric score, but it returns a boolean. Fix: rename it to something that reads as a yes/no question, such as `isStrongPassword` or `isStrong`. No credit for a name that still suggests a number (e.g., `computePasswordStrength`), which is worse than the original.**

    (b) (30pt) The second kind of problem: **Magic numbers. Any one of these, fixed by replacing it with a well-named constant: 12 on line 5 (e.g., `LONG_PASSWORD_LENGTH`); 8 on line 7 (e.g., `MIN_PASSWORD_LENGTH`); 2 on line 6 (e.g., `LONG_PASSWORD_POINTS`); or 2 on line 13 (e.g., `STRONG_SCORE_THRESHOLD`). The two 2s mean different things: deduct points if the answer uses a single constant for both. Other answers may also be acceptable - for example, the structure of the code is strange and could be simplified.**

20. (a) (30pt) Apply the concept of *partition testing* to propose four test cases for `scoreStrength`. Label the different parts of each test case that you propose. **20 points for test cases that clearly split the input space into any four distinct quadrants; the student must show they know that partition testing should result in "intersecting" tests. 10 points for labeling at least the inputs and oracles correctly. Students who define valid partitioned regions in 20(a) rather than full test cases may receive partial credit toward 20(b).**

    (b) (20pt) Explain how you used the concept of partition testing to choose these test cases. **Full credit if the partitioning strategy is sensible and the explanation shows that the student knows what partition testing is. Partitioning strategies that make sense include, but are not limited to: with/without numbers, less than 8/8-11/12+ length.**

A. *No Silver Bullet: Essence and Accidents of Software Engineering*
B. *Syntax, Predicates, Idioms—What Really Affects Code Complexity?*
C. *Relating Reading, Visualization, and Coding for New Programmers: A Neuroimaging Study*
D. *Variability and Reproducibility in Software Engineering: A Study of Four Companies that Developed the Same System*
E. *Hiring is Broken: What Do Developers Say About Technical Interviews?*
F. *The Daikon System for Dynamic Detection of Likely Invariants*
G. *Introduction to TLA*
H. *Build Systems à la Carte*
I. *Expectations, Outcomes, and Challenges of Modern Code Review*
J. *An Experimental Evaluation of Continuous Testing During Development*
K. *Taming Google-Scale Continuous Testing*

IV. "Your Choice" Reading Quiz (30 points)

21. (30pt) Select **one** "Your Choice" reading that you did and circle its letter in the table above. Then, for each of the three claims listed below under that reading's letter, circle **T** if the claim is true of the reading or **F** if it is false. Answer only the claims for the reading that you selected. Scoring: 30 points if all three are correct, and 0 points otherwise. If you didn't do any of the "Your Choice" readings, leave this question blank to receive 6 points.

A. ***No Silver Bullet***

A1 **T** **F** **[F; he is "persuaded that nothing will" come of it]** Brooks predicts that graphical (visual) programming is one of the most promising ways to attack the essential difficulties of software.

A2 **T** **F** **[T; section on time-sharing]** Brooks argues that there is no benefit to making computer response time faster than about 100 milliseconds, the threshold at which humans notice it.

A3 **T** **F** **[F; methodology separates poor from good; great designs come from great designers]** Brooks argues that the difference between good designs and great designs comes mainly from following a sound design methodology.

B. ***Syntax, Predicates, Idioms***

B1 **T** **F** **[T; Sec. 7.3.3]** A condition written as a nested sequence of simple `if`s was not significantly harder to understand than the same condition written as a single compound boolean expression, even though the nested version was much longer.

B2 **T** **F** **[F; Sec. 8: no demographic variable had a significant effect]** More experienced programmers understood the code snippets significantly faster and more accurately than less experienced ones.

B3 **T** **F** **[T; Sec. 7.3.6: 27% longer]** Loops that counted down took noticeably longer to understand than otherwise-identical loops that counted up.

C. ***Relating Reading, Visualization, and Coding for New Programmers***

C1 **T** **F** **[T; Sec. I, III-B]** The researchers measured brain activity with fNIRS (a cap that uses near-infrared light) rather than fMRI.

C2 **T** **F** **[F; 77% were female]** Most of the participants were male.

C3 **T** **F** **[F; coding differed more from reading than from mental rotation]** Like earlier studies of expert programmers, this study found that novices' brain activity while coding closely resembled their activity while reading prose.

D. ***Variability and Reproducibility in Software Engineering***

D1 **T** **F** **[F; Sec. 6.3.1: only one defect was common to two systems]** Most of the defects found after deployment appeared in more than one of the four systems, which suggests that they came from ambiguities in the shared requirements.

D2 **T** **F** **[F; outcomes were not monotonic in price]** The company with the highest price delivered the best system on every quality dimension that was measured.

D3 **T** **F** **[T; Sec. 9.3]** There was no correlation between the schedule each company agreed to and the actual time it took to finish; the company that finished fastest had agreed to the second-longest schedule.

E. ***Hiring is Broken***

E1 **T** **F** **[T; Guideline I]** The authors recommend that phone screens use very easy questions (no harder than the "Rainfall Problem") rather than complex algorithm questions.

E2 **T** **F** **[F; Guideline V says the opposite]** The authors recommend that candidates be interviewed by engineers from outside the team they would join, to reduce bias.

E3 **T** **F** **[F; data was Hacker News comments only]** The authors surveyed hiring managers at companies like Google, Facebook, and Microsoft about how they evaluate candidates.

F. ***The Daikon System***

F1 **T** **F** **[F; also C, C++, Perl, and CSV data; Sec. 3.1]** Daikon can only analyze Java programs.

F2 **T** **F** **[T; Sec. 3.4, 5.2]** Daikon does not report an invariant that is logically implied by other invariants that it reports.

F3 **T** **F** **[T; Sec. 3.4]** Daikon uses a statistical test to avoid reporting properties that might have held only by coincidence, e.g., because a variable was observed only a few times.

G. ***Introduction to TLA***

G1 **T** **F** **[T; section on stuttering]** TLA specifications must allow "stuttering steps," in which the variables that the specification describes do not change.

G2 **T** **F** **[T; section on stuttering]** In TLA, a system that halts is represented by an infinite behavior in which the system's variables stop changing.

G3 **T** **F** **[T; "Implementation is implication"]** In TLA, "specification F implements specification G" means the same thing as "F logically implies G."

H. ***Build Systems à la Carte***

H1 **T** **F** **[T; Sec. 2.2]** The paper treats Microsoft Excel as a build system.

H2 **T** **F** **[F; only proposed; future work, Sec. 8]** The authors implemented Cloud Shake and deployed it at a large company.

H3 **T** **F** **[T; Table 2 and Sec. 5.4; Fabricate: Sec. 7.1]** The build systems that the paper discusses include Buck, Nix, and Fabricate.

I. ***Expectations, Outcomes, and Challenges of Modern Code Review***

I1 **T** **F** **[F; mostly small logic issues: 65 of 78]** Most of the defects that reviewers found were high-level design problems.

I2 **T** **F** **[F; Sec. VII-A: "may be fraught"]** The authors recommend that teams rely on code review as their main quality-assurance mechanism.

I3 **T** **F** **[F; Microsoft teams using CodeFlow]** The authors studied code review in open-source projects such as Apache.

J. ***An Experimental Evaluation of Continuous Testing***

J1 **T** **F** **[T; 50% vs. 27%]** Students given only continuous compilation (without continuous testing) were also more likely to complete the assignment than students with no tool.

J2 **T** **F** **[F; 33% vs. 78%]** Students who ran the tests manually throughout development did about as well as students who had continuous testing.

J3 **T** **F** **[T; Parkinson's Law]** Continuous testing did not significantly change how much time students spent working on the assignment.

K. ***Taming Google-Scale Continuous Testing***

K1 **T** **F** **[F; coverage was infeasible at Google scale; Sec. I]** The authors chose which tests to run using code coverage data collected for each individual test.

K2 **T** **F** **[T; Sec. IV-E]** Files recently modified by two developers caused fewer breakages than files modified by one developer, but files modified by three or more developers caused significantly more.

K3 **T** **F** **[T; "milestones"]** Google's CI system does not test each commit individually: it batches many consecutive commits and tests them together.

V. Document-based Questions (300pts). All questions in this section refer to documents **A** through **D**. These documents appear at the end of the exam (we recommend that you tear them out and refer to them as you answer the questions).

Thief: The Dark Project is a video game developed by Looking Glass Studios and released in late 1998. Documents **A** and **B** are excerpts from a retrospective of the project written by Tom Leonard, who was the game's lead programmer when it shipped. It was published in *Game Developer* magazine in July 1999. Document **A** is the introduction. Document **B** contains seven excerpts, each describing a different aspect of the project. In the original article, some of these excerpts appeared under the heading "What Went Right" and others under "What Went Wrong"; we have removed those headings and shuffled the excerpts. Document **C** is an excerpt from a 2011 article by Sean Barrett, who wrote the game's 3D rendering engine (the code that draws the game's graphics). Document **D** is a timeline of the project that we assembled from these sources.

22. Identify **three** mistakes made by the Thief team that are described in the documents. For each, cite the document (and excerpt number, if applicable), name the most closely-related software engineering concept that we've covered this semester that the mistake relates to, and explain in one sentence what you would have done differently. **Many acceptable answers. Strong ones: the AI and renderer each depended on one person, and nobody else understood them when those people left (B2, B4, B6, C; bus factor / knowledge sharing / documentation); the original AI was built before requirements were specified (B4; requirements); development of the AI continued "without any oversight and without a firm game design" (B4; code review / ownership); the lead delayed acting on the AI's problems despite a colleague's concerns (B4; code review / acting on feedback); the schedule was not re-estimated when it slipped (B7; estimation); the team was staffed before the design was mature, so content was built that was "essentially wrong for the game" (B7; prototyping / requirements); the publisher was shown a series of emergency demos hacked into code that was about to be thrown away, rather than regular demos of real progress (B4; demos / customer communication); the team underestimated the risk of building a new reusable engine at the same time as the game (A: "risks that our team underappreciated"). Do NOT accept on their own: the Object System, scope cuts, or the difficulty system.**

(a) (30pt) The first mistake:

(b) (30pt) The second mistake:

(c) (30pt) The third mistake:

23. (40pt) Excerpt 7 says that the team was "staffed [...] before the design and technology was sufficiently mature," and that as a result the team built "lots of content that was essentially wrong for the game we were making." Relate this to what you learned in this course about requirements and prototyping. What should the team have done instead? **Building before requirements are understood leads to wasted work. Better: a small team builds a prototype (e.g. a single level with the core stealth mechanic) to validate the design with real players and the publisher, and only then scale up content production. This is also exactly what the author says he'll do next time (not shown to students): "a content development team will not be geared up until the technology is sufficiently mature to allow for an informed game design process."**

24. (60pt) Suppose that it is December 1996. You are a programmer at Looking Glass, and your manager has just told you that Sean Barrett is leaving the company in two weeks and that you will be taking over the renderer (Documents **B** and **C**). You may spend as much of those two weeks with Sean as you like. Describe how you would use that time: what would you ask Sean to do or explain, and what would you do yourself? Justify your **two** most important priorities using evidence from the documents. **Open-ended; grade on prioritization and use of evidence. Best answers focus on knowledge that cannot be recovered by reading the code: the techniques "were never written up" (C); the code depends on "a much more complicated set of invariants that had to be maintained" (C), so ask Sean to write those down (and turn them into assertions or tests); known problems (the epsilon/precision problems in the CSG and BSP code, which Sean later says he never solved properly; C); and what work remains (B6: hardware support was added after he left, and the renderer "never received the attention it needed"). Good things to do yourself: read the code with Sean available to answer questions, make a small real change with Sean reviewing it, and get the long document he wrote "over one Christmas vacation" (C) and update it to describe what was actually built. Weaker answers: "ask him to comment every function" (low value per hour), or "convince him to stay" (not in your control, though negotiating a contracting arrangement, which actually happened (B6), is a reasonable plan B).**

    A very common mistake on this question was to answer with something like "ask Sean to describe his design to me", which reproduces the same bus factor problem that already exists (in you); a better answer is to ask him to *document* his design. The rationale for this is the same as the rationale for why you shouldn't explain yourself only in the code review tool in response to a reviewer's question: it isn't discoverable by others in the future.

25. (50pt) Support or refute the following claim: the Thief team's problems with its AI were caused by the departure of the programmer who originally wrote it. **Refute (or heavily qualify). The departure mattered, but Excerpt 4 lists several other causes: the AI was designed "before the requirements of the revised stealth design were fully specified"; its overseer left and most of the programming staff was temporarily reassigned, so it was developed "without any oversight and without a firm game design"; its decision logic had "drifted away from a rigorous design"; and the new lead waited months to decide to rewrite it despite warnings. A codebase with reviews, a spec, and a second person who understood it (i.e., a higher bus factor) would have survived the departure. For full credit, an answer *must* mention at least two out of three of "lack of reviews", "no spec", and "low bus factor"; we frequently gave partial credit for "true but incomplete (TBI)" answers that mentioned only one of the several problematic aspects of the Thief team's engineering practice.**

26. (60pt) Suppose that it is mid-March 1998 and you are Tom Leonard. The team has just decided to throw away four-fifths of the AI code and start over (Excerpt 4). The producer needs to tell the publisher when the new AI will be ready for testing, and asks you for an estimate. Your gut feeling is that it will take about eight weeks. Write the reply (no more than one paragraph) that you would send to the producer. Your reply must include an estimate and explain how you arrived at it. **A professional reply that gives an estimate longer than eight weeks (or a range) and justifies it with evidence gets full credit. Evidence: the team's schedules have repeatedly been "unrealistic" and were not reassessed when they slipped (B7); the Looking Glass systems "take considerable time to develop, do not always arrive on time, and require substantial tuning" (A); the team has just come through months of crunching on demos and "many sleepless nights" (B4); earlier AI work was interrupted by months of emergency demos (B4), so the estimate should state the assumption that there will be no more; and only "the basics of a new design" exist (B4). Strong answers also give a confidence level, propose intermediate checkpoints, and suggest what could be cut if it slips. Partial credit for a bare number, for accepting the gut estimate without justification, or for simply citing the actual outcome (twelve weeks; B4, D) without reasoning that would have been available in March 1998. (A student may fairly cite B7's claim that "late in the winter of 1998, many of our scheduling mistakes had been corrected" as a reason for some confidence.) No more than half credit if the stated answer is less than the twelve weeks that it actually took.**

VI. Extra Credit. Questions in this section do not count towards the denominator of the exam score.

27. (10pt) In section II (Matching), there is a theme to the names used in the situation descriptions. What is the theme? **First names of major-party nominees on the ballot in Newark-area races in the November 3, 2026 general election (US Senate and the House races for NJ-8 through NJ-11): Cory Booker (D) and Justin Murphy (R) for US Senate; Rob Menendez (D, NJ-8); Nellie Pou (D) and Rosie Pino (R) for NJ-9; LaMonica McIver (D) and Carmen Bucco (R) for NJ-10; and Analilia Mejia (D) and Joe Hathaway (R) for NJ-11. Accept any answer that identifies them as candidates in the 2026 midterm elections in New Jersey.**

28. (10pt) What's something you've learned in this class so far that this exam didn't ask you about? **Really anything goes here, as long as it's something that we covered in class. Half-credit for something that was mentioned in class but that's off-topic, like "Professor Kellogg gives us candy when we answer a question." Also half-credit for saying a term we talked about in class (e.g., "mutation testing") without saying anything else about it to show that you'd learned something (if no answer to next question).**

29. (10pt) Propose a multiple-choice question that we could have included in Section I of this exam that tests for whatever you said you learned in question 28. For full credit, you must include at least one *tempting* distractor as a wrong answer—that is, a wrong answer that a student who misunderstood might pick. **Again, anything goes. Must match the previous question. Half-credit if the question is so easy that we wouldn't consider including it on a real exam.**

30. (5pt) If you read one of the "fully-optional" readings linked from the calendar on Labor Day, write anything you remember about it here. **anything true gets credit**

This page intentionally left blank (you may use it as scratch paper, and the proctor will have more scratch paper at the front if you need more).

Document A: Excerpt from Tom Leonard, "Postmortem: *Thief: The Dark Project*," *Game Developer* magazine, July 1999. (Introduction.)

Thief: The Dark Project is one of those games that almost wasn't. During the long struggle to store shelves, the project faced the threat of cancellation twice. A fiscal crisis nearly closed the doors at Looking Glass. During one seven-month span, the producer, project director, lead programmer, lead artist, lead designer and the author of our renderer all left. Worse still, we felt a nagging fear that we might make a game that simply was not fun. But in the end, we shipped a relatively bug-free game that we had fun making, we were proud of, and that we hoped others would enjoy.

[. . .] The project began in the spring of 1996 as "Dark Camelot," a sword-combat action game with role-playing and adventure elements, based on an inversion of the Arthurian legend. Although development ostensibly had been in progress on paper for a year, Thief realistically began early in 1997 after the game was repositioned as an action/adventure game of thievery in a grim fantasy setting. Up to that point we had only a small portion of the art, design, and code that would ultimately make it into the shipping game. Full development began in May 1997 with a team comprised almost entirely of a different group of people from those who started the project. During the following year, the team created a tremendous amount of quality code, art, and design.

But by the beginning of summer in 1998, the game could not be called "fun," the team was exhausted, and the project was faced with an increasingly skeptical publisher. The Looking Glass game design philosophy includes a notion that immersive gameplay emerges from an object-rich world governed by high-quality, self-consistent simulation systems. Making a game at Looking Glass requires a lot of faith, as such systems take considerable time to develop, do not always arrive on time, and require substantial tuning once in place. For Thief, these systems didn't gel until mid-summer, fifteen months after the project began full development, and only three months before we were scheduled to ship.

[. . .] Thief also differed from a number of projects in that it took risks on numerous fronts, risks that our team underappreciated. We wanted to push the envelope in almost every element of the code and design. [. . .] The team was larger than any Looking Glass team up until then, and at times there seemed to be too many cooks in the kitchen. Reaching a point where everyone shared the same vision took longer than expected. A philosophy of creating good, reusable game engine components created unusual challenges that didn't always fit well with schedule and demo pressures.

Document B: Excerpts from Tom Leonard, "Postmortem: *Thief: The Dark Project*," *Game Developer* magazine, July 1999. *(Headings removed and order shuffled by the instructors.)*

Excerpt 1. Our experience on previous titles taught us that one of the impediments to timely game development is the mutual dependence of artists, designers, and programmers at every development stage. One of the development goals for the Dark Engine, on which Thief is built, was to create a set of tools that enabled programmers, artists, and designers to work more effectively and independently. [...] The Dark Object System stood at the heart of our strategy. Primarily designed by programmer Marc "Mahk" LeBlanc, the Object System was a general database for managing the individual objects in a simulation. [...] Using GUI tools, the designers specified the hierarchy and composition of game objects independent of the programming staff. In Thief there was no code-based game object hierarchy of any kind. Although the implementation of the system was much more work than we expected, and management of the object hierarchy placed significant demands on lead designer Tim Stellmach, it turned out to be one of the best things in the project. Once the set of available properties and relations exposed by programmers was mature, the Object System allowed the designers to specify most of the behaviors of the game without scripting or programmer intervention. [...] The data-driven approach worked so well that through much of our development, Thief and System Shock 2 (two very different games) used the same executable and simply chose a different object hierarchy and data set at run time.

Excerpt 2. Midway through 1997, Thief was just starting to gather momentum. We were fully staffed and the stealth design was really starting to get fleshed out. Unfortunately, Looking Glass's financial situation was bleak. Few emotions can compare to the stress of heading to work not knowing who might be laid off, including yourself, or whether the doors would be locked when you got there. The company shed half of its staff in a span of six months [...]. Some of the Thief team couldn't continue under these conditions. We lost two programmers, including the former lead programmer, and a designer. When we were forced to close our Austin office, we lost our producer, Warren Spector, as well as some programmers who made valuable technology contributions to our engine. [...] Although it took some months to fully restore the spirit of the rest of the team, we held together and the company eventually rebounded.

Excerpt 3. Early on, the Thief plan was chock full of features and metagame elements: lots of player tools and a modal inventory user interface to manage them; multiplayer cooperative, death-match and "Theft-match" modes; a form of player extra-sensory perception; player capacity to combine world objects to create new tools; and branching mission structures. These and other "cool ideas" were correctly discarded. Instead, we focused in on creating a single-player, linear, mission-based game centered exclusively around stealth [...]. The notion came into full force with two decisions we made about seven months before we shipped. First, the project was renamed Thief from the working title "The Dark Project," a seemingly minor decision that in truth gave the team a concrete ideological focus. Second, we decided preemptively to drop multiplayer support, not simply due to schedule concerns, but also to allow us as much time as possible to hone the single-player experience. In the end, some missions didn't achieve the stealth focus we wanted, particularly those originally designed for "Dark Camelot," but the overall agenda was the right one.

Excerpt 4. If one thing could be called out as the reason Thief's gameplay didn't come together until late in the process, it would be the AI. The AI as a foil to the player is the central element of Thief, and the AI we wanted wasn't ready until late in the spring of 1998. As lead programmer and author of the final AI, I take full responsibility for that. The original AI for Thief was designed by another programmer before the requirements of the revised stealth design were fully specified. Six months after it was begun, the project director and overseer of the system left the team, and the most of the programming staff was temporarily reassigned to help ship another game that was in trouble. During the following months, development on that AI continued without any oversight and without a firm game design. Soon after, the programmer working on the AI also left. While the core pathfinding data structures and algorithms were basically sound, the code that generated the pathfinding database was extremely buggy. The design of the AI decision process was geared towards an action fighting game requiring little designer customization, rather than a stealth game that needed much more customization. Even worse, the high-level decision process in the AI had drifted away from a rigorous design and the code was extremely brittle. The whole situation was a disaster. These might not have been serious issues, except for one key mistake: I didn't realize the depth of the problem quickly enough, and despite concerns expressed by programmer/designer Doug Church, I didn't act fast enough. I think highly of the programmer involved with the initial AI and wanted to avoid the natural but often misguided programmer reaction within myself that I should just rewrite it my way. So, I took the position that, while buggy, the system as a whole was probably sound. Several months and many sleepless nights later, I concluded that I had been sorely mistaken. By November 1997, I had the basics of a new design and began working on it. But all work had to stop in order to pull together an emergency proof-of-concept demo by the end of December to quell outside concerns that the team lacked a sound vision of the game. This

turned into a mid-January demo, followed by an early February publisher demo, followed by a late February make-or-break demo. During this time the only option was to hack features as best we could into the existing AI. While better than losing our funding, constructing these demos was not good for the project. In the end, work on the new AI didn't begin until mid-March. Despite the fact that our scheduled ship date was just six months away, we threw away four-fifths of our existing AI code and started over. After a hair-raising twelve-week stretch of grueling hours, the AI was ready for real testing. Had I committed to a rewrite two months earlier the previous autumn, I believe the AI would have been ready for real use three to five months sooner.

Excerpt 5. One of the Thief team's favorite games during development was Goldeneye on the N64. We were particularly struck by the manner in which levels of difficulty were handled. Each level of difficulty had a different overlapping set of objectives for success, and missions were subtly changed at each level in terms of object placement and density. Relatively late in the development of Thief, we decided such a system would work well in our game. [. . .] [I]t allowed the designers to create a very different experience at each level of difficulty, without changing the overall geometry and structure of a mission. This gave the game a high degree of replayability at a minimum development cost.

Excerpt 6. The project was started because of the renderer, rather than the reverse. The basic core of the renderer for Thief was written in the fall of 1995 as an after-hours experiment by programmer Sean Barrett. During the following year, the renderer and geometry-editing tools were fleshed out, and with "Dark Camelot" supposed to ship some time in 1997, it looked like we would have a pretty attractive game. Then, at the end of 1996, Sean decided to leave Looking Glass. Although he periodically contracted with us to add features, and we were able to add hardware support and other minor additions, the renderer never received the attention it needed to reach the state-of-the-art in 1998. The possibility that we might not have a point programmer for the renderer weighed heavily on the team. Fortunately, Sean remained available on a contract basis, and other members of the team developed sufficient knowledge of the renderer so that we shipped successfully. In the end, we shipped a renderer appropriate for our gameplay, but not as attractive as other high-profile first-person titles. This may prompt the question of why we didn't simply license a renderer. When the project started a few months into 1996, the avalanche of Quake licenses hadn't really begun and Unreal was still two years away. By the time licensing was a viable choice, the game and the renderer were too tightly integrated for us to consider changing.

Excerpt 7. [. . .] During "Dark Camelot," and continuing through the first half of Thief, we staffed the team before the design and technology was sufficiently mature. In Thief, this led us to rush towards finishing the design, when we didn't necessarily understand the design and technology. With insufficient specifications of both the code systems and mission designs, we ended up doing lots of content that was essentially wrong for the game we were making. Code was written and spaces were built that weren't well directed towards the goals of the project. To make matters worse, we failed to reassess core scheduling assumptions carefully once the schedule began to slip. Captives of a series of unrealistic schedules, we didn't leave enough time for the sort of experimentation, dialogue, and prototyping a project like Thief needs. Late in the winter of 1998, many of our scheduling mistakes had been corrected. Still, during the remainder of the project, the legacy of our earlier missteps required cutting missions that relied on technology we didn't have, and reworking missions not focused on the core gameplay.

Document C: Excerpts from Sean Barrett, "The 3D Software Rendering Technology of 1998's *Thief: The Dark Project*," published on his personal website in 2011.

I was the primary author of the core rendering technology in Thief (although I didn't write the object or character renderers), as well as some related bits and pieces. The same rendering engine, modified by others to use 3D hardware acceleration, also did the rendering for System Shock 2 and Thief 2. [...]

The Quake software rendering was thoroughly documented by Michael Abrash in a series of articles [...]. The techniques used in Thief were never written up and I thought it might be nice to write them down for once, even if they're now totally irrelevant. [...]

[...] I realized that portals could be used to seamlessly integrate multiple independent renderers [...], so I wrote up a long document over one Christmas vacation documenting my thoughts along those lines. Then I guess I had all these ideas bouncing around in my head, and I knew we didn't have any idea how to do non-grid stuff, and it seemed viable to just try to make a full indoor renderer using portals... so I went ahead and tackled implementing one as research. [...]

[...] I'm doubtful there really was a Thief team at that point. [...] In fact I don't know for sure when exactly I did the original research version of this engine. [...]

Partway through Thief development I switched everything in the CSG engine from floats to doubles and cranked down the epsilon, which made things better but didn't solve the problem properly, but I realize in hindsight it would have been much, much better to have simply avoided the BSP tree entirely. The epsilon problems were exacerbated by the crazy way I built the polygons and portals directly from the BSP-tree [...]; this introduced a much more complicated set of invariants that had to be maintained that also had epsilon problems. [...]

Most importantly, Thief used character skinning with skeletal animation to render its characters, and I never touched a single line of code of that system.

I quit Looking Glass for a time; during my absence hardware acceleration support was added. (I came back and added support for colored lighting in the lighting generator and maybe the surface cache or something.)

Document D: Timeline of the project (*assembled by the instructors from Documents A–C and other sources*).

Fall 1995	Sean Barrett writes the core of the renderer as an after-hours experiment.
Spring 1996	The project begins as "Dark Camelot," a sword-combat game.
End of 1996	Barrett leaves Looking Glass. He continues to work on the renderer occasionally as a contractor.
Late 1996 – 1997	Within one seven-month span, the producer, the project director (who oversaw the original AI), the lead programmer, the lead artist, the lead designer, and the renderer's author all leave. The programmer who wrote the original AI leaves soon after.
Early 1997	The game is redesigned as a stealth game about a thief ("The Dark Project").
May 1997	Full development begins, with a team made up almost entirely of different people from those who started the project.
Mid 1997	Looking Glass is in financial crisis and lays off half its staff over six months. Most of the programming staff is temporarily reassigned to help ship another game.
Dec 1997 – Feb 1998	Work on the new AI stops while the team builds a series of emergency demos for the publisher.
Mid-March 1998	Work on the new AI begins. The scheduled ship date is six months away.
April 1998	The game is renamed *Thief: The Dark Project*. Around the same time, multiplayer is cut.
∼June 1998	After twelve weeks, the new AI is ready for testing.
∼August 1998	The game's systems "gel" and it finally becomes fun, three months before the scheduled ship date.
December 1998	The game is released.

H. Honors Section. Only students in the honors section should answer the question in this section. Your grade on it is shifted by *negative* 20 points, but is graded out of 40. So, if you leave it blank, you will get $-20 + (40 * 0.2 =)8 = -12$ points to your overall exam score. If you answer correctly, you score will improve by 20 points. Your exam is still graded out of 800, so you can think of this question as "extra credit with a penalty if you get it wrong or leave it blank."

31. Select **one** "Your Choice" reading from the table at the start of Section IV and write its letter here: _____. It may be the same reading that you chose for question 21 or a different one. You may draw on your reflection essay about this reading, if you wrote one; we *strongly* suggest that you do so..

    (a) (10pt) State one specific finding or claim from the reading, and describe the evidence or argument that the authors give for it. **Must be specific to the reading: a restatement of the title (e.g., "code review is useful") gets no credit. The evidence must match what is actually in the reading.**

    (b) (20pt) Suppose that the study (or argument) were redone today, in 2026. Identify one specific way that software development has changed since the reading was published that is relevant to this finding. Would the finding still hold? Justify your prediction. **Either prediction is acceptable if it is justified. Full credit requires connecting the change to the *reason* that the finding held originally (e.g., its mechanism, subjects, or assumptions), not just asserting that "things are different now."**

    (c) (10pt) Briefly describe how you would design a study to test your prediction: who or what would you study, and what would you measure? **The measurement must actually distinguish between the finding holding and not holding.**